HACKATHON PROJECT DOSSIER & GUIDE

Mayl — Local AI Agent Consent Layer

A human-in-the-loop consent and instant revocation protocol for autonomous open-source AI agents. Powered 100% locally by Qwen2.5 3B, SQLite, and WebGL 3D telemetry.

1. Executive Summary (In Plain English)

Mayl is an open-source software layer that acts like a **digital security guard and passport controller for autonomous AI agents**. Just as a visitor cannot enter an office building without showing an ID badge, receiving temporary visitor credentials, and leaving when their pass expires—an AI agent running under Mayl **cannot perform privileged actions without explicit, time-bounded human consent**, and can be stopped in **less than a second with "one word" (revoke)**.

> **The Core Philosophy:** *"Hire an agent · Bless it with verified consent · Watch it work · Stop it with one word."*
> Mayl replaces blind agent trust with verifiable, cryptographically signed, and local-first consent.

2. Why Should We Use This? (The Real-World Problem)

❌ The Danger with Current AI Agents

- **Blind Autonomy:** Agents run indefinitely with full API access and no expiration.
- **Cloud Lock-in & Data Leaks:** Prompts, keys, and private data are sent to external cloud APIs (OpenAI, Anthropic).
- **No Emergency Brake:** If an agent hallucinates, deletes files, or drains funds, there is no standardized instant halt.
- **Sybil / Bot Spam:** No proof of whether an agent operator is a real human or a malicious botnet.

✅ How Mayl Solves This

- **Time-Bounded Consent Windows:** Permissions automatically expire (e.g., 1 hour, 24 hours).
- **100% Offline & Private:** Uses local **Qwen2.5 3B** via Ollama and a local **SQLite** database. Zero data leaves your machine.
- **Instant 0ms Revoke:** Flipping the app-layer authority flag instantly cuts agent execution.
- **Proof of Personhood:** Integrated with **World ID** orb verification (one verified human = one capped blessing).

3. Key Value Propositions For Hackathon Judges

🔒 100% Offline AI Privacy

Zero cloud dependencies. Powered by local Qwen2.5 3B on Ollama and local SQLite ledger. No API fees, no rate limits, no data leaks.

⚡ Instant One-Word Revoke

App-layer authority check halts execution immediately upon revocation. Cryptographic receipts log every action before and after.

🌐 Web3 Standards Ready

ENSv2 human-readable subnames, World ID zero-knowledge personhood, 1inch Aqua liquidity pool, and ERC-8004 machine pennies.

4. The Mayl Workflow (5-Step Lifecycle)

The entire user journey follows a strict, intuitive state machine where an agent can never dead-end and can never bypass consent:

1 **BROWSE (Agent Discovery & Capability Review)**

The human explores available specialized agents on the shelf (e.g., **Terri the Turtle** for builds, **Trace** for surplus food rescue, **PIT** for compliance audits). Each agent card displays its clear boundary of what it *May* and *May NOT* do.

2 **HIRE (Scope & Time Window Negotiation)**

The user selects an agent and reviews the proposed contract: the specific task, duration limit (e.g., 1 hour), allowed endpoint, and the assigned ENSv2 subname (`agent.mayi.eth`). No privileged code executes yet.

3 **BLESS (The Human Consent Ceremony)**

The human performs the consent ceremony. In production, this uses **World ID** (Orb-grade zero-knowledge proof of human personhood) to guarantee that a genuine human authorizes the agent. Mayl mints a consent session (`cs_...`) in the local SQLite ledger and sets the authority state to **ACTIVE**.

4 **UTILITY (Autonomous Execution via Local Qwen2.5 3B AI)**

The blessed agent executes its tasks. In our implementation, prompts are routed to the offline **Qwen2.5 3B** model. Every single generated output produces an immutable, cryptographically signed receipt stamped with the active session token and logged into SQLite.

5 **REVOKE (The Instant One-Word Halt)**

At any moment, the human can say **"withdraw"** or click the Revoke button. The authority flag flips to **INACTIVE** in 0ms. Any further requests receive a `403 consent_revoked` error. The agent is immediately disarmed.

5. Interactive 3D Cyber-Shield Telemetry

Interactive WebGL 3D Visualization (Three.js):
To give operators real-time visual situational awareness, Mayl renders a 3D holographic Cyber Shield in the browser. It features dual gyroscopic gimbal rings, orbiting sentinel nodes, and a pulsing crystalline AI core. When an agent is blessed, the shield glows electric sky blue (`#38BDF8`). If consent is revoked, the core flashes crimson amber (`#EF4444`) and locks its gimbal rings, giving immediate visual confirmation that the agent is disarmed.

6. Complete Tech Stack: Languages & Tools

Mayl was built strictly from scratch with zero framework bloat, ensuring lightning performance, 100% offline reliability, and instant portability:

Layer	Technology Used	Language	Purpose & Architectural Rationale
Local AI Engine	**Qwen2.5 3B** via **Ollama**	MODEL WEIGHTS	Open-weight 3-billion parameter LLM running locally on CPU/GPU. High-speed reasoning, zero cloud costs, 100% privacy.
Backend Server	Python 3 `ThreadingHTTPServer`	PYTHON 3	Lightweight multi-threaded HTTP daemon (`server.py`) on port 8000. Handles REST APIs, streaming polls, and local Ollama proxy.
Consent Ledger	**SQLite 3** with WAL Mode	SQL / C	Local relational database (`mayi.db`). WAL mode enables non-blocking concurrent reads and writes for chat logs and receipts.
3D Graphics	**Three.js** (WebGL)	JAVASCRIPT	Hardware-accelerated 60 FPS 3D holographic consent shield with mouse parallax, drag-to-rotate, and wireframe toggle.
Frontend UI	HTML5, CSS3, Vanilla JS	HTML/CSS/JS	Glassmorphic Deep Navy + Sky Blue aesthetic. Zero npm build steps; loads instantly on any browser or mobile device.
Web3 Protocols	Ethers.js v6, ENSv2, World ID	SOLIDITY / JS	Sepolia testnet smart contracts: World ID ZK-proofs, ENS subnames, and 1inch Aqua liquidity accounting.

7. Database Schema & Storage Architecture

All data is persistently stored in the local SQLite database `mayi.db` inside the project directory:

📑 Table: `consent_ledger`

Tracks all consent sessions, authorized agents, and revocations:

- `session_id` : Unique consent identifier (`cs_YYYYMMDD...`)
- `agent_subname` : ENSv2 subname (e.g. `terri.mayi.eth`)
- `receipt_token` : Cryptographic bearer token
- `status` : ACTIVE | REVOKED

💬 Table: `chat_messages`

Stores message history with strict sequence order:

- `id` : Auto-increment primary key
- `session_id` : Active visitor chat session UUID
- `seq` : Incremental sequence number (deduplication)
- `role` : `visitor` | `agent` | `system`
- `text` : Message content / AI prompt output
- `created_at` : Timestamp

- `created_at` : ISO-8601 UTC timestamp

8. APIs & Endpoints Detailed Breakdown

Here are the core REST endpoints powered by the Python server and how each one communicates:

Endpoint	Method	Input / Parameters	Action & Return Value
`/send`	POST	`{session, text, name, agent_name}`	Routes prompt to local Qwen2.5 3B, saves user & AI message to SQLite, returns `{ok, visitor_seq, seq, reply}`.
`/poll`	GET	`?session=...&after=...&wait=25`	Long-polling synchronization. Returns all new messages since sequence ID `after`.
`/v1/consent/begin`	POST	`{agent_subname}`	Opens a new consent session in SQLite, generates a receipt token, returns active session ID.
`/v1/revoke`	POST	`{session_id, receipt_token}`	Immediately sets consent status to `revoked` in the database. Disarms the agent in 0ms.
`/healthz`	GET	None	Returns `{"status": "ok", "model": "qwen2.5:3b"}`. Used for service liveness checks.

9. Web3 & Identity Integrations Explained Simply

🌎 World ID (Zero-Knowledge Personhood)

Why it matters: Proves that the operator is a real, unique human being (verified via the Orb) without revealing their personal identity or name. Prevents automated bot swarms from mass-spawning or exploiting agents.

🏷️ ENSv2 (Ethereum Name Service)

Why it matters: Gives every agent a human-readable name like `terri.mayi.eth`. The Standing Consent Window is recorded as a portable on-chain text record, allowing any other dApp to verify if the agent is currently blessed.

🌊 1inch Aqua & PoolGuard

Why it matters: Provides virtual liquidity pool custody where tokens never leave the treasury—the chain only records accounting. One verified human gets one capped share (1%) of the pool.

🪙 ERC-8004 & x402 Machine Pennies

Why it matters: Micro-payment rail using standard HTTP 402 Payment Required headers. An agent performs one discrete job for $0.0042, producing on-chain settlement receipts.

10. Hackathon Mentor & Judge Q&A Guide

Use these concise, authoritative answers during your pitch or when answering questions from the evaluation panel:

❓ Judge: "Why not just use ChatGPT or OpenAI API with function calling?"

Answer: Cloud AI requires sending sensitive prompts and API tokens to third-party servers, creating data leak risks and recurring costs. Furthermore, cloud APIs lack a native, verifiable consent protocol. **Mayl runs 100% locally on open-weight Qwen2.5 3B**, meaning zero data ever leaves the machine, zero API fees exist, and every execution is backed by local cryptographic receipts.

> 💡 *Pitch tip: "Mayl is about AI sovereignty. Your data stays on your machine, and your consent is non-negotiable."*

❓ Judge: "What happens if an AI agent goes rogue or tries to do something unauthorized?"

Answer: Under Mayl, every privileged action checks the app-layer authority flag before execution. If consent has expired or the human types "withdraw" (or clicks Revoke), the authority state flips to inactive immediately (0ms). The agent's execution is immediately blocked with an unbypassable 403 error.

> 💡 *Pitch tip: "It's an emergency brake that cannot be negotiated away by the AI."*

❓ Judge: "Can this system run entirely offline without internet connection?"

Answer: Yes, 100%! The Python server, the SQLite database, the Qwen2.5 3B AI model in Ollama, and the Three.js 3D graphics all run locally on the host machine. You can disconnect Wi-Fi entirely and the complete ceremony, chat, and utility execution will continue functioning seamlessly.

❓ Judge: "How do you prevent malicious bot attacks or mass-spamming agents?"

Answer: Through **World ID Orb-grade Proof of Personhood**. The protocol enforces that only verified unique human operators can bless agents or claim pool allocations. Wallets without verified personhood are rejected with a `401 personhood_required` status.

11. 30-Second Elevator Pitch For Hackathon Presentation

> **"Judges, AI agents are taking over automation, but today they operate with either zero boundaries or blind trust. Mayl solves this by introducing a Local-First Consent Layer for autonomous agents. You hire an agent, bless it with timed permissions backed by verified human personhood, watch it run 100% offline via local Qwen2.5 3B AI, and if anything goes wrong, you stop it in zero milliseconds with one word: Revoke. Private, auditable, and human-controlled. Thank you!"**